

PENETRATION TESTING REPORT

MEDIROZA GENERAL HOSPITAL

- **Client:** Mediroza General Hospital
- **Target:** <https://medirozahospital.com>
- **Assessment Type:** Black-Box Web Application Penetration Test
- **Prepared By:** Tanjan Singh Karki — Networkwalks, Batch B083
- **Authorization:** Written authorization was provided by the client for the assessment.
- **Classification:** **CONFIDENTIAL — Client and Authorized Personnel Only**

1. EXECUTIVE SUMMARY

A black-box penetration test was conducted against the publicly accessible web application of Mediroza General Hospital to identify security vulnerabilities and assess their potential impact.

The assessment identified multiple weaknesses affecting authentication, sensitive file exposure, directory configuration, and information disclosure.

The most significant findings were:

- **SQL Injection** in the patient portal login, allowing authentication bypass without valid credentials.
- **Public exposure of a database backup** containing sensitive employee and shareholder information.
- **Directory listing** on sensitive application directories, making internal files easier to discover.
- **Sensitive path disclosure through robots.txt**, which assisted reconnaissance.
- **Document metadata disclosure** that identified an internal IT administrator account name in a patient report.

The findings demonstrate that an unauthenticated attacker could potentially progress from publicly available information to unauthorized application access and access to highly sensitive information.

RISK OVERVIEW

ID	Finding	Severity
F1	SQL Injection — Authentication Bypass	Critical
F2	Publicly Exposed Database Backup	Critical
F3	Directory Listing on Sensitive Paths	Medium
F4	Sensitive Paths Disclosed via robots.txt	Low
F5	Document Metadata / Access-Control Indicator	Medium

The assessment particularly highlights the need for stronger **secure coding practices, access control, backup management, and web-server hardening**.

2. SCOPE & METHODOLOGY

2.1 SCOPE

In Scope

- Target: <https://medirozahospital.com>
- Publicly accessible web application
- Authentication mechanisms
- Public files and directories
- Application input handling
- File-access functionality

Out of Scope

- Social engineering
- Denial-of-service testing
- Physical security
- Internal network infrastructure
- Systems outside the authorized target domain

2.2 METHODOLOGY

Testing followed a structured black-box methodology:

1. Passive reconnaissance
2. Web application enumeration
3. Public file and directory discovery
4. Manual application analysis
5. Authentication testing
6. SQL injection testing
7. Sensitive file-access testing
8. Document analysis and metadata inspection
9. Impact assessment
10. Remediation recommendations

2.3 TOOLS & TECHNIQUES

- Web browser and Developer Tools
- Manual HTTP/application testing
- SQL injection payload testing
- Directory and file enumeration
- Networkwalks Hash Calculator & Password Cracker
- Custom password wordlist
- PDF metadata inspection

2.4 ASSESSMENT LIMITATIONS

The assessment was performed without privileged credentials or internal documentation. Testing was limited to the authorized web application and did not include infrastructure, physical, or internal-network testing.

3. FINDINGS SUMMARY

ID	Vulnerability	Severity	Primary Impact
F1	SQL Injection — Authentication Bypass	Critical	Unauthorized patient portal access
F2	Sensitive Database Backup Exposed	Critical	Exposure of employee and corporate data
F3	Directory Listing Enabled	Medium	Sensitive file and application discovery
F4	Sensitive Paths in robots.txt	Low	Reconnaissance assistance
F5	Document Metadata Leakage	Medium	Internal account/process disclosure

4. DETAILED FINDINGS

F1 — SQL INJECTION: AUTHENTICATION BYPASS

Severity: Critical

Location: <https://medirozahospital.com/patient/login.php>

Category: Injection / Authentication Bypass

Description

The patient portal login functionality was found to incorporate user-controlled input directly into a database query without adequate parameterization.

This allowed SQL injection to alter the intended authentication query and bypass password validation.

Proof of Concept

Username:

admin' --

Password:

Any value

Observed Result: Authentication was successfully bypassed and access to the patient portal was obtained.

Impact

An unauthenticated attacker could potentially:

- Bypass patient portal authentication.
- Access functionality intended for authenticated users.
- Access confidential patient documents exposed through the portal.
- Increase the risk of unauthorized disclosure of protected health information.

Root Cause

User-controlled input was incorporated into a SQL statement without using parameterized queries or prepared statements.

Remediation

- Use parameterized queries / prepared statements for all database operations.
- Implement strict server-side input validation.
- Review all application endpoints for similar SQL injection patterns.
- Use a database account with the minimum required privileges.
- Implement appropriate authentication and session controls.
- Conduct security-focused code review and regression testing.

F2 — SENSITIVE DATABASE BACKUP PUBLICLY EXPOSED

Severity: Critical

Location: /old/mediroza_db_backup_2019.sql

Category: Sensitive Data Exposure / Security Misconfiguration

Description

A historical database backup was publicly accessible through the web server.

The file was located within a web-accessible directory and could be downloaded without authentication.

Exposed Information

The backup contained sensitive information including:

- Employee names
- Job titles
- Departments
- Email addresses
- Phone numbers
- National identification numbers
- Salary information
- Shareholder names
- Share ownership percentages
- Shares held
- Share classes

Impact

Public exposure of the backup creates significant confidentiality risks, including:

- Employee privacy violations
- Potential identity theft
- Exposure of payroll information
- Corporate ownership disclosure
- Increased risk of targeted attacks

Root Cause

An old database backup was stored within the publicly accessible webroot.

Remediation

1. Remove the exposed backup immediately.
2. Store backups outside the webroot.
3. Restrict backup access using appropriate permissions.

4. Encrypt backups at rest.
5. Implement secure backup retention and deletion procedures.
6. Search the entire webroot for additional backup files such as:
 - .sql
 - .bak
 - .zip
 - .tar
 - .gz
7. Review web-server access logs to determine whether the file was accessed by unauthorized parties.

F3 — DIRECTORY LISTING ENABLED ON SENSITIVE PATHS

Severity: Medium

Locations:

/old/

/patient/

/staff/

Category: Security Misconfiguration

Description

Directory indexing was enabled on multiple sensitive application directories.

Accessing these directories returned an Index of listing containing files and subdirectories.

Observed Exposure

/old/

- mediroza_db_backup_2019.sql

/patient/

- download.php
- error_log
- login.php
- logout.php
- portal.php
- reports/

/staff/

- login.php

Impact

Directory listing reduces the effort required to discover application components and sensitive files.

In this assessment, directory listing directly assisted in discovering the exposed database backup.

Root Cause

Directory browsing/indexing was enabled at the web-server configuration level.

Remediation

- Disable directory indexing.
- Return 403 Forbidden or an appropriate response for restricted directories.
- Remove unnecessary files from the webroot.
- Review existing directories for sensitive artifacts.
- Restrict access to administrative and application-specific resources.

F4 — SENSITIVE PATHS DISCLOSED VIA ROBOTS.TXT

Severity: Low

Location: /robots.txt

Category: Information Disclosure

Description

The publicly accessible robots.txt file disclosed several sensitive application paths:

Disallow: /patient/

Disallow: /staff/

Disallow: /old/

robots.txt is designed to provide instructions to search-engine crawlers. It does **not** provide access control.

Impact

The disclosure itself presents limited risk; however, it provided useful reconnaissance information and helped identify locations that were subsequently found to contain accessible resources.

Root Cause

Sensitive directories were treated as locations to hide from indexing rather than being protected through actual authorization controls.

Remediation

- Do not use robots.txt as a security mechanism.
- Enforce authentication and authorization at the application/server level.
- Avoid unnecessarily disclosing sensitive paths.
- Review the content of robots.txt for information that could assist attackers.

F5 — DOCUMENT METADATA LEAKAGE / ACCESS-CONTROL INDICATOR

Severity: Medium

Location: Retrieved patient laboratory reports

Category: Information Disclosure

Description

Metadata was examined across three retrieved patient laboratory report PDFs.

Two documents identified **Mediroza Hospital** as the author, consistent with system-generated reports.

A third document contained the author value: jmalik

This corresponded to the account name associated with the hospital's IT Systems Administrator identified during the assessment.

Impact

The metadata does not independently provide unauthorized access. However, the account-name disclosure may reveal information about internal document handling and administrative identities.

The difference in document authorship also warrants internal review to determine whether the document was generated or modified outside the standard clinical reporting workflow.

Root Cause

Document metadata was not standardized or sanitized before delivery.

Remediation

- Review access permissions for patient-report generation and modification.
- Restrict administrative access to patient records.
- Enable audit logging for access and modification of patient documents.
- Standardize or remove unnecessary document metadata.
- Investigate the identified document's creation and modification history.

Note: Metadata alone does not establish unauthorized access or inappropriate activity. The observed discrepancy should be validated through internal audit logs and document-generation records.

5. RISK ASSESSMENT

The severity ratings consider:

- Authentication requirements
- Exploit complexity
- Confidentiality impact
- Data sensitivity
- Ease of discovery
- Potential downstream impact

Finding Severity Rationale

F1	Critical	Unauthenticated SQL injection allowed authentication bypass and access to protected application functionality.
F2	Critical	Public database backup exposed highly sensitive employee and corporate information.
F3	Medium	Directory indexing significantly improved discovery of sensitive application resources.
F4	Low	robots.txt provided reconnaissance information but does not itself enforce or bypass access control.
F5	Medium	Metadata disclosed an internal account identifier and indicated a potentially important document-handling issue requiring validation.

6. ATTACK CHAIN OBSERVED

The assessment demonstrated how individually different weaknesses could contribute to a broader compromise path:

Public Reconnaissance

|



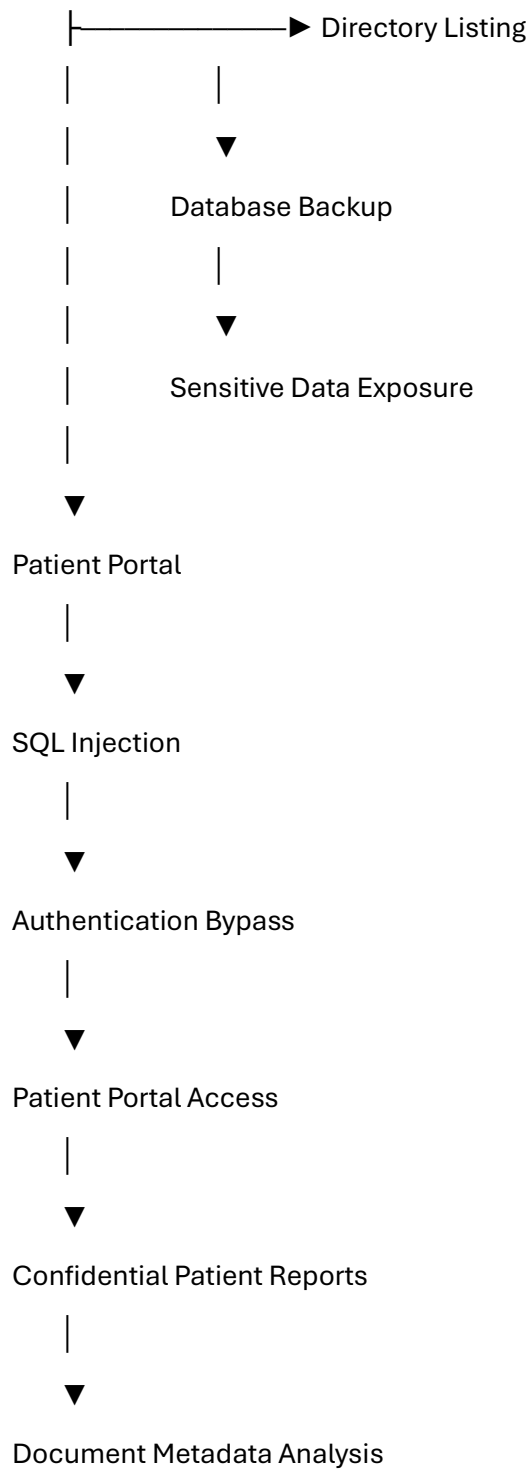
robots.txt

|



Sensitive Directory Discovery

|



This demonstrates the importance of addressing both **individual vulnerabilities and the relationships between them.**

7. REMEDIATION PLAN

Immediate Priority

1. Remove the exposed database backup

- Delete the publicly accessible backup.
- Move legitimate backups outside the webroot.
- Review server logs for previous access.
- Assess whether exposed information requires incident-response action.

2. Fix SQL Injection

- Replace dynamic SQL construction with prepared statements.
- Review all authentication and input-handling code.
- Perform regression testing after remediation.

3. Review Exposed Patient Data

- Verify which patient documents were accessible.
- Review access logs.
- Confirm appropriate authorization controls for patient records.

Short-Term Hardening

- Disable directory indexing.
- Review /old/, /patient/, and /staff/.
- Remove unused scripts, logs, backups, and temporary files.
- Review web-server permissions.
- Standardize document metadata.
- Implement stronger audit logging.

Long-Term Improvements

- Establish secure software-development practices.
- Conduct regular vulnerability assessments and penetration tests.
- Implement secure backup-management procedures.
- Provide secure coding training focused on injection vulnerabilities.
- Apply least-privilege principles to application and database accounts.
- Consider a WAF as an additional defense-in-depth control.

8. CONCLUSION

The assessment identified several security weaknesses within the Mediroza General Hospital web application.

The most significant issues were the **SQL injection vulnerability in the patient portal** and the **publicly accessible database backup**. Together with directory listing and information-disclosure issues, these weaknesses substantially increased the potential for unauthorized access and sensitive-data exposure.

Priority should be given to:

1. Removing exposed sensitive files.
2. Correcting SQL injection vulnerabilities.
3. Reviewing patient-data access controls.
4. Disabling directory indexing.
5. Strengthening backup and document-management practices.
6. Conducting a broader security review to identify similar weaknesses elsewhere in the application.

Remediation should be followed by **verification testing** to confirm that the identified vulnerabilities have been successfully addressed.

9. ASSESSMENT INFORMATION

Field	Details
Client	Mediroza General Hospital
Target	https://medirozahospital.com
Assessment	Black-Box Web Application Penetration Test
Tester	Tanjan Singh Karki
Program	Networkwalks — Batch B083
Authorization	Written authorization provided
Primary Focus	Web Application Security
Overall Risk	Critical
Report Classification	Confidential